



Afleveringsopgave #2 – Teknisk MVP
REST API Microservice løsning

Fag: Integration, Cloud, DevSecOps & Automation

Uddannelse: Økonomi og IT

Navn: Yassmin Salma El Kounti

Dato: 03.05.2026

Problem og domænekobling

Denne opgave tager udgangspunkt i domænet Predictive Maintenance inden for Internet of Things (IoT). I dette domæne anvendes sensorer til kontinuerligt at overvåge maskiners tilstand med henblik på at identificere potentielle fejl, før de udvikler sig til kritiske nedbrud.

Predictive Maintenance er et centralt forretningsområde, da det gør det muligt at reducere uplanlagt nedetid, optimere vedligeholdelsesomkostninger og forbedre driftssikkerheden. I stedet for reaktiv vedligeholdelse, hvor fejl først håndteres efter de opstår, muliggør predictive maintenance en proaktiv tilgang baseret på data.

I denne løsning behandles sensordata som events. Når en sensor sender en temperaturmåling, opfattes dette som et domæneevent ("Sensor Value Received"). Systemet analyserer herefter data og træffer en beslutning baseret på en regel.

Hvis temperaturen overstiger en fastsat threshold, klassificeres det som en anomaly ("Anomaly Detected"), hvilket kan indikere en potentiel fejl.

Dette skaber en direkte kobling mellem forretningsbehov og teknisk implementering, hvor realtidsdata anvendes til beslutningstagning.

Løsningsdesign og arkitektur

Systemet er implementeret som en REST API baseret microservice. Arkitekturen er designet med fokus på enkelhed, men følger centrale principper fra moderne softwarearkitektur, herunder separation of concerns og event-driven design.

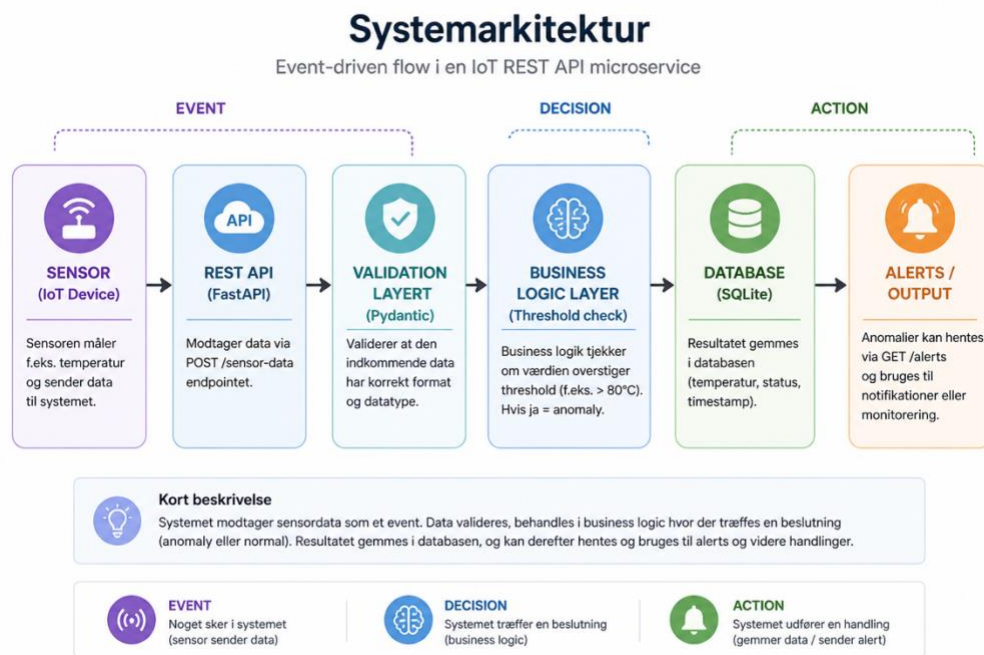
Systemets komponenter:

- Sensor (IoT device)
- REST API (FastAPI)

- Validation layer (Pydantic)
- Business logic layer
- Database (SQLite)
- Output (alerts)

Dataflow:

Sensor → REST API → Validation → Business Logic → Database → Response



Når systemet modtager et event via API'et, gennemgår data flere lag:

1. **API layer** modtager data
2. **Validation layer** sikrer korrekt struktur
3. **Business logic layer** udfører beslutninger
4. **Database layer** gemmer resultatet
5. **Response layer** returnerer output

Arkitekturen følger et event-driven flow, hvor systemet reagerer på indkommende data og transformerer det til en handling.

Denne opdeling gør systemet mere modulært og skalerbart.

Implementering

Løsningen er udviklet i Python med FastAPI, som er et moderne framework til udvikling af REST API'er.

Teknologier:

- FastAPI → API udvikling
- Pydantic → datavalidering
- SQLite → datalagring

Endpoint struktur:

POST /sensor-data

Dette endpoint modtager sensordata i JSON format. Data valideres automatisk via Pydantic, hvilket sikrer korrekt datatype og struktur.

Når data er valideret, sendes det videre til business logic laget.

Business logic:

Systemet anvender en regelbaseret tilgang:

Hvis temperatur > 80 → anomaly

Ellers → normal

Denne regel repræsenterer systemets beslutningslogik.

Persistens:

Data gemmes i en SQLite database, hvor både temperatur og status (normal/anomaly) registreres.

GET /alerts

Dette endpoint returnerer alle anomalies fra databasen, hvilket muliggør overvågning og analyse.

Event-driven flow og teknisk forståelse

Systemet er baseret på et event-driven design.

Event:

Sensor sender data til systemet

Decision:

Systemet analyserer data via business logic

Action:

Resultatet gemmes og kan hentes via API

Dette flow er centralt i moderne systemer, hvor systemer reagerer dynamisk på data i realtid.

Event-driven arkitektur anvendes ofte i IoT løsninger, da det muliggør fleksibel og skalerbar behandling af store datamængder.

DevSecOps og deployment

Løsningen er udviklet og testet lokalt ved hjælp af uvicorn, som starter API serveren.

Versionstyring kan foretages via GitHub, hvilket gør det muligt at tracke ændringer og samarbejde effektivt.

CI/CD pipelines kan implementeres via GitHub Actions eller Azure DevOps, hvor kode automatisk buildes, testes og deployes.

Deployment kan ske i cloud-miljøer som Microsoft Azure, hvor løsningen kan skaleres efter behov.

Containerisering via Docker kan anvendes til at gøre deployment mere konsistent og portabel.

Kvalitet og drift

Systemet indeholder flere kvalitetsmekanismer:

- Input validering via Pydantic
- Simpel fejlhåndtering via API responses
- Test via FastAPI interface

Systemet returnerer klare responses, hvilket gør det lettere at debugge og overvåge.

I en produktionsløsning ville man implementere:

- Logging
- Monitoring
- Error tracking

Dette sikrer stabil drift og hurtig fejlidentifikation.

Refleksion og videreudvikling

Hvad fungerede godt:

Systemet er simpelt, men funktionelt og demonstrerer tydeligt event → decision → action.

Arkitekturen er overskuelig og let at forstå.

Hvad kunne forbedres:

Anomaly detection er baseret på en statisk threshold, hvilket gør systemet mindre fleksibelt.

Videreudvikling:

Systemet kan forbedres ved at implementere machine learning til anomaly detection.

I stedet for faste regler kan systemet trænes på historiske data og identificere afvigelser automatisk.

Dette vil gøre systemet mere intelligent og realistisk i forhold til industrielle løsninger.